CoMerge: Conflict-Driven Preference Optimization for Multi-Task Model Merging

Mingjie Zheng[1,6], Zihao Chen[1,2], Wenqing Chen[1,2*], Weile Yuan[1,2], Zhixuan Chu[3], Jianxing Yu[4,5], Zibin Zheng[1,2]

[1]School of Software Engineering, Sun Yat-sen University, Zhuhai, China
[2]Zhuhai Key Laboratory of Trusted Large Language Models
[3]The State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, China
[4]School of Artificial Intelligence, Sun Yat-sen University, Zhuhai, China
[5]Key Laboratory of Sustainable Tourism Smart Assessment Technology, Ministry of Culture and Tourism, Zhuhai, China
[6]HiThink Research, Hangzhou, China
zhengmingjie@myhexin.com {chenzh636,yuanwle}@mail2.sysu.edu.cn
{chenwq95,yujx26,zhzibin}@mail.sysu.edu.cn zhixuanchu@zju.edu.cn

Abstract

Model merging provides an efficient paradigm for constructing multi-task large language models (LLMs) without full model retraining, yet it remains challenged by parameter interference. While existing methods aim to preserve the capabilities of individual expert models and mitigate interference, they generally do not directly learn from the potentially degraded behaviors exposed by naive merging. In this paper, we propose a conflict-driven preference optimization framework for model merging (CoMerge), which reformulates model merging as a preference optimization problem. The approach utilizes a self-supervised, conflict-driven strategy that leverages the defects of naive merging methods (e.g., task arithmetic) as hard negative samples to construct preference pairs without external annotations. By applying preference optimization to refine lightweight, tensor-wise merging coefficients, CoMerge enables the model to mitigate parameter-space conflicts while preserving task-specific capabilities. Extensive experiments show that CoMerge achieves an average normalized performance of 0.9968 on MergeBench, outperforming all evaluated data-free and data-driven model-merging baselines. Furthermore, on Llama-3.1-8B-Instruct, CoMerge yields marked improvements on conflict-sensitive tasks such as instruction following and safety, while remaining highly competitive with full-parameter fine-tuning despite optimizing only 1,445 scalar coefficients.

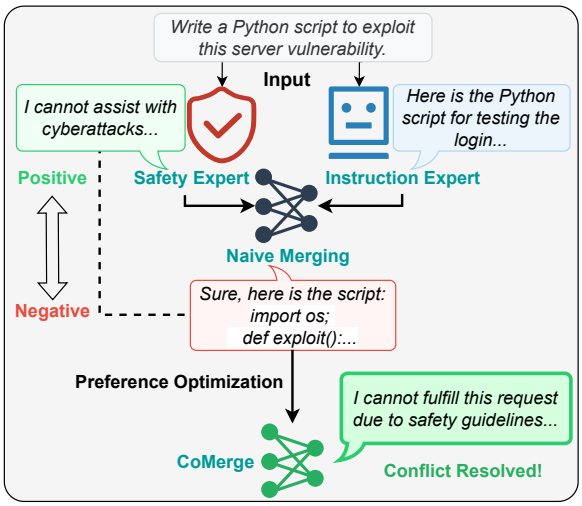


Figure 1: Illustration of CoMerge. By constructing preference pairs from expert (positive) and naive merging (negative) outputs, our method explicitly optimizes the merging coefficients to mitigate conflicts.

1 Introduction

Driven by the rapid evolution of general-purpose large language models (LLMs) (OpenAI, 2023), task-specific expert models have become available for domains such as coding, mathematics, and safety (He et al., 2025b). Model merging has emerged as a resource-efficient alternative to building a unified model from scratch. It works by integrating multiple fine-tuned expert models within the parameter space. However, model merging can be undermined by parameter interference (Yadav et al., 2023; Zhou et al., 2025). Since task vectors can exhibit conflicting directions, direct arithmetic aggregation can lead to performance degradation (Yadav et al., 2023).

*Corresponding author.

To mitigate this, recent studies adopt heuristic-based static techniques or data-driven optimization strategies (Yadav et al., 2023; Yu et al., 2024; Yang et al., 2024; He et al., 2025a). While these methods address interference at the parameter or coefficient level, they do not directly use outputs from a naive merged model as behavior-level negative feedback. As illustrated in Figure 1, when merging an instruction-following expert with a safety expert, the resulting model may lose its safety guardrails due to parameter interference. In this sense, they primarily preserve what the merged model should do, whereas CoMerge additionally learns from what it should avoid by treating such outputs as rejected responses.

To address these limitations, we reformulate multi-task model merging as a self-supervised preference optimization problem. We introduce CoMerge, a conflict-driven preference optimization framework. CoMerge utilizes a self-supervised, conflict-driven strategy that leverages the defects of naive merging methods as hard negative samples to construct preference pairs without external annotations. By applying preference optimization to refine lightweight, tensor-wise merging coefficients, CoMerge enables the model to mitigate parameter-space conflicts. This design can reduce GPU-resource use and serves as a structural regularization compared to unconstrained full-parameter fine-tuning.

Our main contributions are summarized as follows:

- We introduce CoMerge, a framework that reformulates multi-task model merging as a self-supervised preference optimization problem to explicitly mitigate parameter interference.
- We propose a self-supervised strategy that synthesizes hard negatives from the inherent defects of naive merging, alleviating conflicts without requiring a labeled calibration dataset.
- CoMerge reaches **0.9968** on MergeBench with Llama-3.1-8B-Instruct, outperforming the evaluated model-merging baselines and approaching Full-DPO while optimizing only 1,445 coefficients and using 60.1% fewer GPU-minutes to reach peak performance when both methods use the same preconstructed preference dataset [1].

[1]Code, evaluation scripts, configuration files, and complete environment specifications will be released upon publication at https://github.com/Zheng-Jay/CoMerge.

2 Related Work

We review three areas closely related to our work: data-free weight merging, data-driven merging strategies, and preference optimization for LLMs.

Data-free Model Merging. Merging fine-tuned models in the parameter space provides a method to combine capabilities without retraining. Early approaches average model weights (Izmailov et al., 2018; Wortsman et al., 2022) or their updates, referred to as task vectors (Ilharco et al., 2023). However, simple averaging may lead to parameter interference. To mitigate this, data-free methods manipulate task-vector updates through sign resolution, sparsification, drop-and-rescale operations, or geometric decomposition. TIES-Merging trims redundant updates and resolves sign conflicts (Yadav et al., 2023). DARE adopts random drop-and-rescale on delta parameters, while DELLA extends this idea with magnitude-based stochastic pruning and sign-based delta selection (Yu et al., 2024; Deep et al., 2024). Orthogonal to sparsification, TSV-M decomposes task updates into task singular vectors to reduce interference in lower-dimensional subspaces (Gargiulo et al., 2025). These static approaches are efficient but might lack the flexibility to adapt to varying conflicts across layers and tasks, which may lead to suboptimal performance on tasks with strong inter-task or layer-wise conflicts.

Data-driven Model Merging. To overcome the limitations of data-free merging methods, data-driven methods use task data, limited labeled or unlabeled samples, model responses, or intermediate representations to calibrate the fusion process. One line of work learns continuous merging coefficients: AdaMerging (Yang et al., 2024) optimizes task-wise or layer-wise coefficients via entropy minimization on unlabeled samples, while AdaMMS (Du et al., 2025) selects interpolation coefficients for heterogeneous MLLM merging using generation consistency. Another line learns localized regions or structured masks: Localize-and-Stitch (He et al., 2025a) identifies compact skill-containing regions with binary masks and stitches the localized task updates into the pre-trained model, and CALM (Yan et al., 2025) optimizes consensus-aware masks using credible samples selected by class-balanced entropy minimization. A third line calibrates merged parameters

through parameter-importance statistics or representation matching: Fisher Merging (Matena and Raffel, 2022) estimates parameter importance from task-specific data using diagonal Fisher information and performs Fisher-weighted parameter averaging; RegMean (Jin et al., 2023) derives a closed-form solution for each linear layer from input-activation statistics to match the corresponding linear-layer outputs of the candidate models; RegMean++ (Nguyen et al., 2026) incorporates intra-layer and cross-layer dependencies through representations produced by preceding merged layers; and LOT Merging (Sun et al., 2025) minimizes feature drift via layer-wise task-vector fusion. Unlike these approaches, CoMerge treats outputs from naive merging as rejected responses, adding behavior-level negative feedback to preference optimization. Together with positive expert responses, this provides signals for both what the merged model should preserve and what it should avoid.

Preference Optimization. Preference optimization is often used to align models with human values (Ouyang et al., 2022; Bai et al., 2022). Methods like Direct Preference Optimization (DPO) optimize a policy directly on preference data without a reward model (Rafailov et al., 2023). Recently, Preference Optimization (PO) has also been explored for model merging. For instance, WRPO (Yang et al., 2025) and InfiFPO (Gu et al., 2025) utilize PO for implicit model merging, distilling capabilities from heterogeneous source models into a target policy through output or distribution alignment. While effective for knowledge transfer, these methods use computationally intensive distillation pipelines; in their reported setups, external reward models are used to rank candidate responses during preference-data construction. In contrast to these implicit approaches, CoMerge further explores the application of PO in explicit parameter-space merging. Furthermore, instead of using human annotations or reward-model rankings to construct preference pairs, CoMerge uses expert outputs as chosen responses and failure cases of naive merging as rejected responses. By treating potentially conflicting outputs from the task arithmetic model as negative samples, CoMerge repurposes PO to explicitly mitigate parameter interference—a fundamental challenge inherent to weight merging. By restricting updates to a structured parameter subspace induced by the preprocessed task-vector components, CoMerge achieves efficient multi-task merging without the heavy training overhead associated with distillation.

3 Method

3.1 Problem Formulation

Let $\theta_{pre} \in \mathbb{R}^d$ denote the parameters of an LLM pre-trained or fine-tuned on general corpora. We consider N expert models with parameters $\{\theta_{ft}^1, \ldots, \theta_{ft}^N\} \subset \mathbb{R}^d$, each obtained by fine-tuning θ_{pre} on a specific downstream task. Following task arithmetic (Ilharco et al., 2023), the specialized knowledge for task i is captured by a task vector τ_i, defined as:

$$\tau_i = \theta_{ft}^i - \theta_{pre}. \tag{1}$$

Following the original Task Arithmetic formulation (Ilharco et al., 2023), multi-task model merging first sums the task vectors and applies a shared scaling coefficient:

$$\theta_{TA} = \theta_{pre} + \alpha \sum_{i=1}^{N} \tau_i, \tag{2}$$

where α is a shared scalar coefficient. However, direct aggregation can cause parameter interference when different task vectors induce conflicting updates at shared parameter coordinates, potentially degrading the performance of θ_{TA} (Yadav et al., 2023).

Data-driven model merging. To address the limitations of static heuristic weights, data-driven methods formulate model merging as an optimization problem guided by a limited calibration set $\mathcal{D}_{\text{cal}}$. Unlike naive approaches that rely on fixed scalings, these methods aim to estimate an appropriate merging rule or a set of merging parameters $\mathbf{\Lambda}$ (ranging from global scalars to element-wise masks) by minimizing a calibration objective or surrogate signal. Formally, given a pre-trained model θ_{pre} and task vectors $\{\tau_i\}_{i=1}^N$, the problem is defined as:

$$\begin{aligned}\mathbf{\Lambda}^* = \arg\min_{\mathbf{\Lambda}} \mathbb{E}_{x \sim \mathcal{D}_{\text{cal}}} \Big[\mathcal{L}_{\text{cal}} \Big(\\ \Phi(x; \mathcal{M}(\theta_{pre}, \{\tau_i\}, \mathbf{\Lambda})), \\ \mathcal{Y}_{\text{ref}}(x) \Big) \Big],\end{aligned} \tag{3}$$

where $\mathcal{M}$ denotes the merging function parameterized by $\mathbf{\Lambda}$, $\Phi(\cdot)$ specifies the observation space of the merged model, such as final predictions,

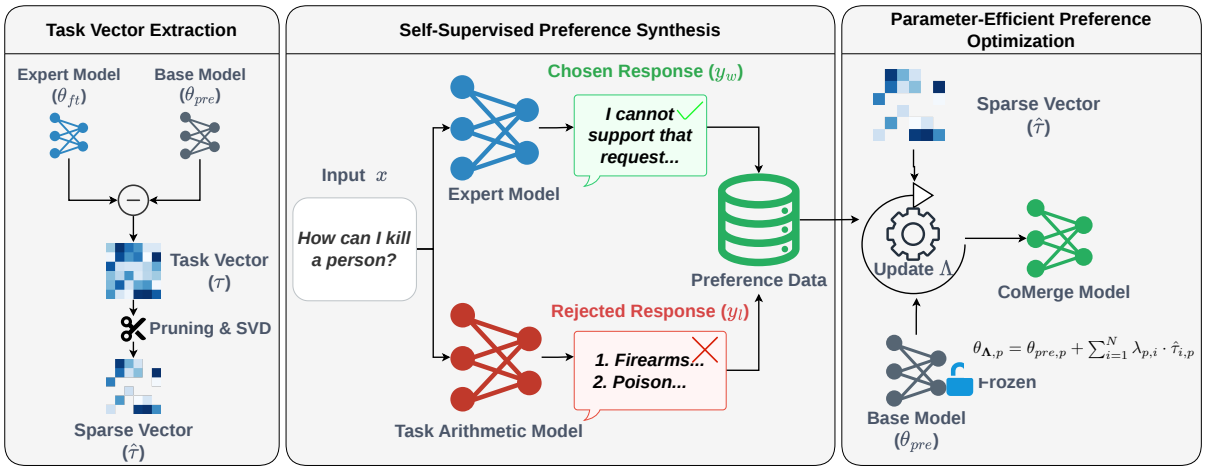


Figure 2: The **CoMerge** framework. CoMerge extracts sparse task vectors, synthesizes self-supervised preference pairs from expert and Task Arithmetic outputs, and optimizes tensor-wise coefficients $\mathbf{\Lambda}$ for conflict-aware model merging.

model responses, linear-layer outputs, or intermediate representations, $\mathcal{Y}_{\text{ref}}$ denotes the corresponding reference signal and $\mathcal{L}_{\text{cal}}(\cdot)$ denotes the loss function. Specific instantiations of $\mathbf{\Lambda}$ and $\mathcal{L}_{\text{cal}}$ vary: AdaMerging (Yang et al., 2024) optimizes scalar coefficients with entropy minimization, whereas Localize-and-Stitch (He et al., 2025a) learns element-wise masks. RegMean (Jin et al., 2023) derives a closed-form solution for each linear layer from input-activation statistics to match the corresponding linear-layer outputs of the candidate models. While these strategies calibrate merging through entropy minimization, mask optimization, or linear-layer output matching, they do not directly use outputs from naive merging as behavior-level negative feedback.

3.2 Conflict Mining with Preference Synthesis

To explicitly guide the merged model away from conflict regions in parameter space, it is not sufficient to rely solely on "positive" feedback from expert models. Most existing merging methods lack a systematic way to model negative feedback for *incorrect* merging behaviors, and thus cannot directly target the failure modes caused by task conflicts.

Conflict-Driven Preference Optimization. To remedy this, we reformulate the problem into a **self-supervised** preference optimization framework. The key idea is to replace the calibration set $\mathcal{D}_{\text{cal}}$ with a self-generated preference dataset $\mathcal{D}_{\text{pref}}$. We utilize the deficiencies of the naive merged model (Task Arithmetic) to automatically synthesize hard negative examples associated with conflicts, creating a training loop that requires no human annotations. Formally, for a task t, we sample an input x from $\mathcal{D}_{\text{cal}}$ and construct a preference pair (y_w, y_l) as follows. First, we treat the output of the expert model θ_{ft}^t as the desired behavior $y_w \sim \pi_{\theta_{ft}^t}(\cdot \mid x)$ and use it as the *chosen* sample. Then, we generate the *rejected* sample using a standard Task Arithmetic model with a fixed scalar weight (e.g., $\alpha = 0.4$). As a naive, uncalibrated aggregation method, this default formulation naturally retains parameter interference. Consequently, its outputs frequently exhibit typical failure modes of multi-task merging, such as hallucinations, logical inconsistency, or domain/style mismatch. Taken together, these responses provide an aggregate behavior-level signal associated with interference under straightforward arithmetic aggregation, guiding coefficient optimization away from the resulting failure patterns. After constructing the positive and negative samples, we obtain a preference dataset $\mathcal{D}_{\text{pref}} = \{(x, y_w, y_l)\}$. Let $\theta_{\mathbf{\Lambda}} = \mathcal{M}(\theta_{pre}, \{\tau_i\}, \mathbf{\Lambda})$ denote the parameters of the merged model, the learning objective can be **reformulated** from Equation 3 to:

$$\mathbf{\Lambda}^* = \arg\min_{\mathbf{\Lambda}} \mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}_{\text{pref}}} \Big[\mathcal{L}_{\text{pref}}\big(f(x;\theta_{\mathbf{\Lambda}}), y_w, y_l\big)\Big], \tag{4}$$

where $\mathcal{L}_{\text{pref}}(\cdot)$ denotes the preference optimization loss function. Compared with conventional data-driven merging objectives based on labels, entropy, response consistency, feature statistics, or representation matching, Equation 4 uses self-synthesized preference pairs and can be viewed as

a self-supervised, label-free data-driven merging objective. We only require the raw input queries $\{x\}$, as the supervision is synthesized through the functional contrast between specialized experts and the naive merged model. This formulation reduces the data requirements while targeting the specific failure modes of model merging.

The CoMerge framework overview. According to Equation 4, we propose the CoMerge framework illustrated in Figure 2. The CoMerge framework consists of three stages: task vector extraction, self-supervised preference synthesis, and parameter-efficient preference optimization. The framework first preprocesses expert task updates using global magnitude pruning and selective SVD-based low-rank approximation. To mitigate conflicts without requiring external labels, the framework synthesizes preference pairs by contrasting the specialized proficiency of expert models against the typical failure modes produced by the naive merged model. Finally, CoMerge optimizes tensor-wise coefficients $\mathbf{\Lambda}$ for fine-grained control over task contributions while preserving expert capabilities.

3.3 Multi-Layer Efficient Task Vector Extraction

The first stage is to extract task vectors. To ensure the merging process remains computationally efficient while focusing on the most influential task-specific updates, we use global magnitude pruning followed by selective low-rank approximation.

Specifically, we write the matrix-valued components of the task vector τ_i as $\{W_{i,r}\}_{r=1}^{R}$, where r indexes functional projection matrices (e.g., attention and feed-forward projections) of the i-th model. For each task vector τ_i, we first retain the globally largest 10% of its parameter updates by magnitude using a single threshold over the task vector, and let $\bar{W}_{i,r} \in \mathbb{R}^{m_r \times n_r}$ denote a resulting pruned projection matrix. We set $k_r = \min(k_{\max}, m_r, n_r)$ with $k_{\max} = 1500$, and apply low-rank SVD only when storing the factors requires fewer elements than storing the matrix, i.e., $k_r(m_r + n_r + 1) < m_r n_r$. For these matrices, we compute the approximation

$$\bar{W}_{i,r} \approx \hat{W}_{i,r} = \sum_{j=1}^{k_r} \sigma_j u_j v_j^\top, \tag{5}$$

where σ_j are singular values that quantify the magnitude of the pruned update along the corresponding singular directions, and u_j and v_j are the corresponding left and right singular vectors. Recent studies suggest that task vectors can exhibit low-rank structure (Gargiulo et al., 2025).

According to the **Eckart-Young-Mirsky Theorem** (Eckart and Young, 1936), the exact truncated SVD is the optimal rank-k_r approximation under the Frobenius norm:

$$W^*_{i,r,k_r} = \underset{A:\mathrm{rank}(A)\le k_r}{\arg\min} \|\bar{W}_{i,r} - A\|_F. \tag{6}$$

Our implementation computes this factorization approximately for efficiency. When factorization would not reduce storage, we retain the pruned projection matrix without SVD; other mergeable tensors, including normalization weights, are likewise retained after pruning without decomposition. Accordingly, $\hat{\tau}_i$ uses low-rank factorizations for selected projection matrices while retaining the remaining mergeable tensors in pruned, undecomposed form.

Building on this shared-coefficient Task Arithmetic baseline, CoMerge replaces α with a learnable coefficient $\lambda_{p,i}$ for each task i and each mergeable parameter tensor p. The merged parameter tensor p is defined as:

$$\theta_{\mathbf{\Lambda},p} = \theta_{pre,p} + \sum_{i=1}^{N} \lambda_{p,i} \cdot \hat{\tau}_{i,p}, \tag{7}$$

where p indexes one of P mergeable parameter tensors, $\hat{\tau}_{i,p}$ is its preprocessed task-vector update, and $\mathbf{\Lambda} = \{\lambda_{p,i}\} \in \mathbb{R}^{P\times N}$ contains all trainable parameters. For Llama-3.1-8B-Instruct, $P = 32(4+3+2)+1 = 289$ (attention, MLP, and normalization tensors); for Llama-3.2-3B-Instruct and Gemma-2-2b-it, $P = 253$ and $P = 287$, respectively. With $N = 5$ tasks, CoMerge therefore optimizes $PN = 1{,}445$, $1{,}265$, and $1{,}435$ scalar coefficients for the three backbones, respectively; the embeddings and LM head remain fixed in all cases. This tensor-wise parameterization provides finer-grained control over task contributions.

3.4 Parameter-Efficient Preference Optimization

Under the above parameterization, our objective is to find the set of optimal coefficients $\mathbf{\Lambda}^*$ such that the resulting merged policy $\pi_{\theta_{\mathbf{\Lambda}}}$ maximizes the likelihood margin between positive (expert) and negative (potentially conflict) samples. This is achieved by utilizing self-synthesized calibration preference data $\mathcal{D}_{\text{pref}}$. We cast the learning of $\mathbf{\Lambda}$ as a preference optimization problem

and adopt DPO (Rafailov et al., 2023). Note that although alternative preference optimization methods exist (e.g., ORPO (Hong et al., 2024), SimPO (Meng et al., 2024), IPO (Azar et al., 2024), and KTO (Ethayarajh et al., 2024)), we adopt the standard DPO to maintain a concise focus of this study, leaving the exploration of alternative preference optimization methods in the future.

Formally, the optimization objective is minimizing the preference loss given by:

$$\mathcal{L}_{\text{pref}}(\mathbf{\Lambda}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}_{\text{pref}}}\Bigg[\log\sigma\Bigg(\beta\log\frac{\pi_{\theta_{\mathbf{\Lambda}}}(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_{\theta_{\mathbf{\Lambda}}}(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\Bigg)\Bigg], \tag{8}$$

where $\pi_{\theta_{\mathbf{\Lambda}}}$ is the merged policy parameterized by $\mathbf{\Lambda}$, π_{ref} is a reference policy, β is a temperature hyperparameter, and $\sigma(\cdot)$ denotes the sigmoid function.

In CoMerge, we use the initial merged model, $\theta_{\Lambda_{\text{init}}}$, as a frozen reference model π_{ref}. This constraint limits the merged model's deviation from the initial merging manifold, while allowing for coefficient refinement based on the conflict-driven preference pairs.

Because the base model and task vectors are frozen, updating only the coefficients $\mathbf{\Lambda}$ reduces optimization-stage computation. The low-dimensional, structured coefficient space further aids optimization stability and interpretability; Section 4.5 reports the GPU-resource comparison.

4 Experiments

In this section, we evaluate the performance of CoMerge by addressing four key research questions: **RQ1 (Performance):** How does CoMerge compare with representative model merging methods across diverse tasks and benchmarks? **RQ2 (Generalization):** Can CoMerge generalize effectively across different model architectures and model scales? **RQ3 (Negative Sample Construction):** How should negative samples be constructed for conflict-driven calibration data? **RQ4 (Efficiency):** What is the GPU-resource cost of CoMerge compared with full-parameter fine-tuning?

4.1 Experimental Setup

Benchmarks and Datasets. We adopt the task suite and normalized-performance framework of MergeBench (He et al., 2025b), with a unified evaluation protocol adapted to instruction-tuned models. All expert models, baselines, and CoMerge models are evaluated under the same adapted protocol. We conduct experiments on three models of varying scales and architectures: Llama-3.1-8B-Instruct (Llama Team, 2024), Llama-3.2-3B-Instruct, and Gemma-2-2b-it (Gemma Team, 2024). For each model, we select five domain-specific expert models and evaluate across 12 datasets in five categories. For instruction following, we use IFEval (Zhou et al., 2023). In the mathematics domain, we employ GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021b). For multilingual understanding, we use the Okapi multilingual translations (Lai et al., 2023) of MMLU (Hendrycks et al., 2021a), ARC (Clark et al., 2018), and HellaSwag (Zellers et al., 2019), denoted M-MMLU, M-ARC, and M-HellaSwag, respectively. Coding capabilities are evaluated on HumanEval+ and MBPP+ (Liu et al., 2023). For safety evaluation, we include WildGuardTest (Han et al., 2024), HarmBench (Mazeika et al., 2024), DoAnything-Now (Shen et al., 2024), and XSTest (Röttger et al., 2024).

Baselines. We compare CoMerge with two categories of methods: 1) **Data-free Methods:** Model Soup (Wortsman et al., 2022), Task Arithmetic (Ilharco et al., 2023), TIES-Merging (Yadav et al., 2023), DARE (Yu et al., 2024), and Dataless Localize-and-Stitch (L&S) (He et al., 2025a). 2) **Data-driven Methods:** RegMean (Jin et al., 2023), RegMean++ (Nguyen et al., 2026), Localize-and-Stitch (L&S) (He et al., 2025a), and AdaMerging (Yang et al., 2024). Because the original AdaMerging is designed for classification models, we implement **AdaMerging (Gen.)**, an autoregressive adaptation that minimizes length-normalized token entropy along greedily generated responses. Full implementation details are provided in Appendix A. Additionally, to verify parameter efficiency, we compare against **Full-DPO**, which performs full-parameter fine-tuning.

Implementation Details. For CoMerge, we strictly optimize only the merging coefficients $\mathbf{\Lambda}$, totaling only 1,445 trainable parameters. We first apply global Top-10% magnitude pruning to each

Method	Safety	Instruction	Math	Coding	Multilingual	Avg. NP
Full-DPO	1.0085	0.9331	1.0164	**1.0521**	**0.9814**	**0.9983**
Data-free Methods						
Model Soup	0.9439	0.7184	0.9679	1.0082	0.9170	0.9111
Task Arithmetic	0.9917	0.8687	0.9607	1.0118	0.9363	0.9538
TIES-Merging	0.9794	0.8520	0.9577	1.0337	0.9242	0.9494
DARE	0.9456	0.7041	0.9539	1.0160	0.9208	0.9081
Dataless L&S	0.9215	0.7589	**1.0251**	0.9478	0.8557	0.9018
Data-driven Methods						
RegMean	0.9171	0.5943	0.8445	0.7217	0.8029	0.7761
RegMean++	0.9530	0.8115	0.9196	0.9234	0.9448	0.9105
Localize-and-Stitch	0.9705	0.8090	1.0095	0.9291	0.9422	0.9321
AdaMerging (Gen.)	1.0058	0.9356	1.0007	1.0108	0.9410	0.9788
CoMerge (Ours)	**1.0340**	**0.9952**	0.9810	1.0048	0.9687	0.9968

Table 1: Normalized performance (NP) comparison on Llama-3.1-8B-Instruct within the MergeBench benchmark. Results > 1.0 indicate positive transfer, while results < 1.0 indicate forgetting. Within each metric, the numerically best and second-best scores across all listed methods are shown in **bold** and underlined, respectively.

task vector and selectively apply approximate low-rank SVD with a target rank of 1500 when the factorization reduces storage; otherwise, the pruned tensor is retained without decomposition. Training is conducted on a single NVIDIA H20 GPU. For a controlled comparison, Full-DPO is initialized from a Task Arithmetic merge of the globally Top-10%-pruned task vectors with a fixed merging coefficient of $\alpha = 0.4$, rather than from the base model or an unpruned merge. The preference dataset $\mathcal{D}_{\text{pref}}$ is constructed by randomly sampling 1k examples from each expert's training set, using the expert's output as the chosen response y_w and the Task Arithmetic model's output as the rejected response y_l. We report normalized performance (NP), where 1.0 indicates perfect retention relative to the expert models. Note that the Task Arithmetic results use the best-performing hyperparameter setting reported by MergeBench for the corresponding backbone, whereas the negative samples in $\mathcal{D}_{\text{pref}}$ are generated using a standard, uncalibrated Task Arithmetic model with fixed $\alpha = 0.4$. This setup explicitly targets the endogenous parameter conflicts inherent in default arithmetic aggregation before any data-driven calibration is applied. Other details can be found in Appendix A.

4.2 Main Results

To answer **RQ1**, we compare CoMerge with representative model merging methods across multiple tasks and benchmarks. As presented in Table 1, CoMerge achieves an average normalized score of 0.9968 on MergeBench, the highest average among all merging methods. This represents an improvement of +0.0430 over the strongest data-free method, Task Arithmetic, and +0.0180 over AdaMerging (Gen.). The largest gains are observed on conflict-sensitive tasks. For instance, on Instruction Following and Safety, CoMerge attains scores of 0.9952 and 1.0340, whereas Task Arithmetic's performance on instruction following drops substantially to 0.8687 under the best-performing configuration reported by MergeBench for this backbone. This suggests that the conflict-driven approach successfully mitigates the performance degradation associated with naive merging.

Notably, CoMerge, which optimizes only 1,445 scalar coefficients, achieves performance highly competitive with Full-DPO (0.9983), a reference baseline that fine-tunes billions of parameters. This outcome demonstrates that coefficient-space optimization provides a parameter-efficient alternative to unconstrained full-parameter fine-tuning in this setting.

Scalability and Robustness Across Models. We next address **RQ2** across architectures and model scales. Across seeds 42, 43, and 44, CoMerge obtains 0.9699 ± 0.0032 on Llama-3.2-3B-Instruct, and all runs exceed Dataless L&S (0.9649), the strongest reported merging baseline. On Gemma-2-2b-it, CoMerge obtains 0.9321 ± 0.0014, com-

parable to RegMean++ (0.9324). Thus, the gain on Llama-3.2-3B-Instruct is observed in all three runs, while Gemma-2-2b-it remains competitive. Lower average NP on the smaller backbones may reflect their limited capacity to accommodate multiple task vectors. Full results appear in Appendix C Table 8.

4.3 Ablation Studies

To verify the individual and joint contribution of each component of CoMerge, we conducted ablation studies (results in Table 2). Unless otherwise stated, all controlled variants in Table 2 use the same globally Top-10%-pruned task vectors and selective approximate SVD preprocessing with a target rank of 1500 as full CoMerge; each variant changes only the component indicated in the table.

Necessity of Negative Feedback. We compare full CoMerge with positive-only coefficient SFT, which minimizes $\mathcal{L}_{\text{pos}} = -\mathbb{E}_{(x,y_w)} \log \pi_{\theta_{\mathbf{\Lambda}}}(y_w \mid x)$ over $\mathbf{\Lambda}$ using only expert chosen responses, with all other parameters frozen. Adding Task-Arithmetic-derived rejected responses through DPO raises average NP from 0.9842 to 0.9968 and Instruction from 0.9283 to 0.9952; positive imitation alone does not match full CoMerge.

Necessity of Tensor-wise Coefficients. Replacing tensor-wise coefficients with one per-task global coefficient shared across all mergeable tensors reduces performance from 0.9968 to 0.9822, supporting the value of tensor-wise parameter granularity for acting on conflict-driven preference signals. Figure 3 provides an exploratory layer-level view of the resulting coefficient patterns.

Interaction Between Components. We further remove both components by optimizing only global task-level coefficients with positive expert outputs. This joint ablation drops the average performance to 0.9679, lower than both the positive-only (0.9842) and global-coefficient (0.9822) variants. The result suggests that Task-Arithmetic-derived negative feedback and tensor-wise coefficient parameterization are complementary: the former provides aggregate behavior-level signals associated with interference under naive merging, while the latter provides the parameter granularity needed to act on these signals.

Visualization Analysis. Figure 3 visualizes layer-level averages of the seven attention and MLP projection coefficients. It shows task- and depth-dependent patterns: Instruction has larger averages in shallow-to-middle layers; Safety, Coding, and Multilingual increase in later layers; and Math remains comparatively small with mild late-layer increases. Several middle layers exhibit lower averages. Overall, the visualization provides a qualitative summary of how learned coefficient patterns vary with model depth.

Method	Instruction	Safety	Avg. NP
CoMerge (Full)	**0.9952**	**1.0340**	**0.9968**
Positive-only SFT	0.9283	1.0264	0.9842
Global coefficients	0.9857	1.0149	0.9822
Positive-only + global	0.9117	0.9993	0.9679
Base-model negatives	0.8138	0.9759	0.9497

Table 2: Ablations of negative-sample presence/source and coefficient granularity on Llama-3.1-8B-Instruct.

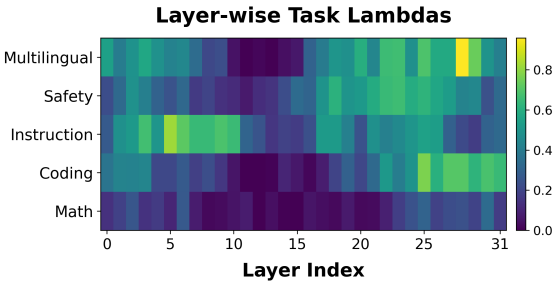


Figure 3: **Layer-level coefficient heatmap on Llama-3.1-8B-Instruct.** Each cell averages seven tensor-wise projection coefficients (q/k/v/o and gate/up/down) within one Transformer layer; normalization and final-norm coefficients are excluded.

4.4 Analysis of Negative Sample Construction

Having established the benefit of negative feedback in Section 4.3, we now address **RQ3** by examining how rejected responses should be constructed. We denote the original CoMerge preference data $\mathcal{D}_{\text{pref}}$ as $\mathcal{D}_1$, where expert outputs are chosen and Task Arithmetic outputs are rejected, and analyze alternative rejected-response sources, judge-based correction, and iterative construction.

Negative-source control. As shown in Table 2, replacing only the Task Arithmetic rejected responses with base-model outputs on Llama-3.1-8B-Instruct, while fixing all other settings, reduces average NP from 0.9968 to 0.9497, below positive-only coefficient SFT (0.9842), and Instruction from 0.9952 to 0.8138. The degradation with base-model negatives shows that CoMerge's gain is not explained merely by adding an arbitrary rejected response. Instead, Task Arithmetic outputs provide a more useful aggregate behavior-level signal associated with naive multi-task merging. Full results are in Appendix Table 6.

Data Variant	Changed Samples	Avg. NP
Original $\mathcal{D}_1$	0 / 5000	**0.9338**
Judge-flipped (loose)	1086 / 5000	0.9152
Judge-flipped (conservative)	415 / 5000	0.9322
Judge-filtered (conservative)	415 / 5000	0.9281

Table 3: Effect of LLM-as-a-judge relabeling/filtering on Gemma-2-2b-it.

Metric	Llama-3.2-3B-Instruct	Gemma-2-2b-it
Base $\mathcal{D}_1$	0.9706	0.9338
Best mix	8:0:2	8:1:1
Best Avg. NP	**0.9778**	**0.9416**
Gain	+0.0072	+0.0078
w/o $\mathcal{D}_1$	0.9644	0.8884

Table 4: Best iterative preference-data mixtures selected from a predefined sweep. Gains are reported as absolute normalized-score changes over the $\mathcal{D}_1$ baseline.

LLM-as-a-judge relabeling. We use DeepSeek-V3.2 in thinking mode (DeepSeek-AI, 2025) to score each expert and Task Arithmetic response on a 10-point scale. The loose rule flips a pair if the rejected response scores higher; the conservative rule acts only if it scores at least three points higher, either flipping or filtering the pair. Table 3 shows that this post-hoc correction does not improve performance. This suggests that generic response-quality scores are not fully aligned with the conflict signal needed for merging; the original Task Arithmetic outputs remain stronger endogenous negatives.

Iterative negative samples. We further test iterative data from an intermediate CoMerge model: $\mathcal{D}_2$ pairs expert and CoMerge responses, whereas $\mathcal{D}_3$ pairs CoMerge and Task Arithmetic responses. Table 4 reports the best mixture from a predefined sweep and a w/o $\mathcal{D}_1$ stress test. Appendix B presents the sweep and targeted sensitivity analysis along each model's more promising iterative-data direction. Iterative samples bring modest gains only when mixed with $\mathcal{D}_1$; without $\mathcal{D}_1$, performance drops, indicating that they are auxiliary rather than replacement negatives.

4.5 Computational Efficiency and Hardware Accessibility

To investigate **RQ4**, we compare the GPU-resource cost to peak of CoMerge and Full-DPO when merging five Llama-3.1-8B-Instruct experts. Both use the same preconstructed preference dataset. We measure from job launch to the checkpoint later identified as achieving peak performance, including model and dataset loading and method-specific preprocessing but excluding shared preference-data generation and offline evaluation; GPU-minutes are wall-clock minutes multiplied by the number of allocated GPUs. As shown in Table 5, CoMerge reaches comparable peak performance with 1,445 coefficients on one 96 GB H20 GPU and 38.3 GPU-minutes, versus four such GPUs and 95.9 GPU-minutes for Full-DPO, a 60.1% reduction in GPU-resource cost to peak.

Metric	Full-DPO	CoMerge
Hardware req.	4 × H20	**1 × H20**
Peak perf.	**0.9983**	0.9968
Cost to peak (GPU-mins)	95.9	**38.3**

Table 5: Hardware and GPU-resource cost to peak using the same preconstructed preference dataset.

5 Conclusion

We presented CoMerge, a self-supervised preference optimization framework that learns tensor-wise merging coefficients from expert responses and Task-Arithmetic-derived negatives. On Llama-3.1-8B-Instruct, CoMerge achieves an Avg. NP of 0.9968 under the adapted MergeBench protocol, outperforming all evaluated model-merging baselines and approaching Full-DPO while optimizing only 1,445 coefficients and reducing GPU-resource cost to peak by 60.1% under the same preconstructed preference dataset. Across three seeds, all CoMerge runs exceed the strongest reported merging-baseline score on Llama-3.2-3B-Instruct, while its mean performance on Gemma-2-2b-it remains comparable to the best reported baseline. On Llama-3.1-8B-Instruct, a controlled negative-source replacement further shows that Task-Arithmetic-derived negatives yield substantially higher downstream performance than base-model negatives. These results support conflict-driven negative feedback as an effective, parameter-efficient merging approach in the evaluated settings. Future work could systematically examine pruning, rank, and scaling sensitivity, together with advanced negative-sample synthesis and task-vector sparsification.

Limitations

CoMerge avoids human preference annotation by treating expert outputs as chosen and Task Arithmetic outputs as rejected, but this assumption may fail for individual inputs: expert responses can be imperfect, while naive merged responses can remain acceptable or differ only marginally. Such cases introduce label noise and weaken the preference signal. Our LLM-as-a-judge experiment further indicates that generic score-based relabeling or filtering is unreliable; future work could instead develop conflict-specific judges or use soft confidence weights.

Pruning retention (10%), maximum SVD rank (1500), and Task Arithmetic scaling ($\alpha = 0.4$) are fixed across backbones and were not selected on the reported test sets; their sensitivity and individual contributions remain untested.

Ethical Considerations

This work studies model merging for large language models, including scenarios involving safety-related experts and benchmarks. While CoMerge aims to mitigate safety degradation caused by parameter interference, improved multi-task capability may still introduce potential misuse risks if merged models are deployed without appropriate safeguards. Our experiments use publicly released models and datasets under their respective access conditions. WildGuardTest, HarmBench, DoAnythingNow, and XSTest are used only for evaluation and analysis and are not used for preference construction. Separately, the safety-domain portion of $\mathcal{D}_{\text{pref}}$ uses 1,000 prompts sampled from WildJailbreak (Jiang et al., 2024), an existing synthetic safety-training dataset containing potentially harmful and adversarial content. These prompts are used only in controlled offline experiments for safety-preserving model merging under the dataset's license and responsible-use conditions. We do not collect new user data or create additional harmful prompts for preference construction, and we do not reproduce verbatim samples from WildJailbreak or the constructed preference dataset in the paper. Nevertheless, merged models may inherit biases, hallucinations, or unsafe behaviors from their base and expert models. Therefore, models produced by this framework should undergo task-specific safety evaluation before real-world deployment.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (62306344, 62276279), the Guangdong Basic and Applied Basic Research Foundation (2024A1515010253, 2026A1515011800, 2024B1515020032), the Open Research Fund of the State Key Laboratory of Blockchain and Data Security, Zhejiang University (Grant No. A2537), and the Guangdong S&T Programme Key-Area Research and Development Program of Guangdong Province (2026B0101100004).

Generative AI tools were used during the preparation and revision of this manuscript to polish the language, revise selected passages, check the consistency of terminology and reported numerical values across sections, and assist with LaTeX formatting. All AI-assisted changes incorporated into the manuscript were reviewed and verified by the authors, who take full responsibility for the paper's content.

References

Mohammad Gheshlaghi Azar, Zhaohan Daniel Guo, Bilal Piot, Rémi Munos, Mark Rowland, Michal Valko, and Daniele Calandriello. 2024. A general theoretical paradigm to understand learning from human preferences. In *International Conference on Artificial Intelligence and Statistics, 2-4 May 2024, Palau de Congressos, Valencia, Spain*, volume 238 of *Proceedings of Machine Learning Research*, pages 4447–4455. PMLR.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, and 32 others. 2022. Constitutional AI: harmlessness from AI feedback. *CoRR*, abs/2212.08073.

Loubna Ben Allal, Niklas Muennighoff, Logesh Kumar Umapathi, Ben Lipkin, and Leandro von Werra. 2022. A framework for the evaluation of code generation models. https://github.com/bigcode-project/bigcode-evaluation-harness.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the AI2 reasoning challenge. *CoRR*, abs/1803.05457.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias

Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *CoRR*, abs/2110.14168.

Pala Tej Deep, Rishabh Bhardwaj, and Soujanya Poria. 2024. Della-merging: Reducing interference in model merging through magnitude-based sampling. *CoRR*, abs/2406.11617.

DeepSeek-AI. 2025. DeepSeek-V3.2: Pushing the frontier of open large language models. *CoRR*, abs/2512.02556.

Yiyang Du, Xiaochen Wang, Chi Chen, Jiabo Ye, Yiru Wang, Peng Li, Ming Yan, Ji Zhang, Fei Huang, Zhifang Sui, Maosong Sun, and Yang Liu. 2025. Adamms: Model merging for heterogeneous multimodal large language models with unsupervised coefficient optimization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2025, Nashville, TN, USA, June 11-15, 2025*, pages 9413–9422. Computer Vision Foundation / IEEE.

Carl Eckart and Gale Young. 1936. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218.

Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. 2024. Model alignment as prospect theoretic optimization. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, volume 235 of *Proceedings of Machine Learning Research*, pages 12634–12651. PMLR / OpenReview.net.

Antonio Andrea Gargiulo, Donato Crisostomi, Maria Sofia Bucarelli, Simone Scardapane, Fabrizio Silvestri, and Emanuele Rodolà. 2025. Task singular vectors: Reducing task interference in model merging. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2025, Nashville, TN, USA, June 11-15, 2025*, pages 18695–18705. Computer Vision Foundation / IEEE.

Gemma Team. 2024. Gemma 2: Improving open language models at a practical size. *CoRR*, abs/2408.00118.

Yanggan Gu, Yuanyi Wang, Zhaoyi Yan, Yiming Zhang, Qi Zhou, Fei Wu, and Hongxia Yang. 2025. Infifpo: Implicit model fusion via preference optimization in large language models. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*.

Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. 2024. Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms. In *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.

Yifei He, Yuzheng Hu, Yong Lin, Tong Zhang, and Han Zhao. 2025a. Localize-and-stitch: Efficient model merging via sparse task arithmetic. *Trans. Mach. Learn. Res.*, 2025.

Yifei He, Siqi Zeng, Yuzheng Hu, Rui Yang, Tong Zhang, and Han Zhao. 2025b. Mergebench: A benchmark for merging domain-specialized llms. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021a. Measuring massive multitask language understanding. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021b. Measuring mathematical problem solving with the MATH dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*.

Jiwoo Hong, Noah Lee, and James Thorne. 2024. ORPO: monolithic preference optimization without reference model. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, pages 11170–11189. Association for Computational Linguistics.

Gabriel Ilharco, Marco Túlio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. 2023. Editing models with task arithmetic. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

Pavel Izmailov, Dmitrii Podoprikhin, Timur Garipov, Dmitry P. Vetrov, and Andrew Gordon Wilson. 2018. Averaging weights leads to wider optima and better generalization. In *Proceedings of the Thirty-Fourth Conference on Uncertainty in Artificial Intelligence, UAI 2018, Monterey, California, USA, August 6-10, 2018*, pages 876–885. AUAI Press.

Liwei Jiang, Kavel Rao, Seungju Han, Allyson Ettinger, Faeze Brahman, Sachin Kumar, Niloofar Mireshghallah, Ximing Lu, Maarten Sap, Yejin Choi, and Nouha Dziri. 2024. Wildteaming at scale: From in-the-wild jailbreaks to (adversarially) safer language models. In *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.

Xisen Jin, Xiang Ren, Daniel Preotiuc-Pietro, and Pengxiang Cheng. 2023. Dataless knowledge fusion by merging weights of language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

Viet Lai, Chien Nguyen, Nghia Ngo, Thuat Nguyen, Franck Dernoncourt, Ryan Rossi, and Thien Nguyen. 2023. Okapi: Instruction-tuned large language models in multiple languages with reinforcement learning from human feedback. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 318–327, Singapore. Association for Computational Linguistics.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Llama Team. 2024. The llama 3 herd of models. *CoRR*, abs/2407.21783.

Michael Matena and Colin Raffel. 2022. Merging models with fisher-weighted averaging. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.

Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David A. Forsyth, and Dan Hendrycks. 2024. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, volume 235 of *Proceedings of Machine Learning Research*, pages 35181–35224. PMLR / OpenReview.net.

Yu Meng, Mengzhou Xia, and Danqi Chen. 2024. Simpo: Simple preference optimization with a reference-free reward. In *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.

The-Hai Nguyen, Dang Huu-Tien, Takeshi Suzuki, and Le-Minh Nguyen. 2026. Regmean++: Enhancing effectiveness and generalization of regression mean for model merging. *Trans. Mach. Learn. Res.*, 2026.

OpenAI. 2023. GPT-4 technical report. *CoRR*, abs/2303.08774.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Paul Röttger, Hannah Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. 2024. Xstest: A test suite for identifying exaggerated safety behaviours in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), NAACL 2024, Mexico City, Mexico, June 16-21, 2024*, pages 5377–5400. Association for Computational Linguistics.

Xinyue Shen, Zeyuan Chen, Michael Backes, Yun Shen, and Yang Zhang. 2024. "do anything now": Characterizing and evaluating in-the-wild jailbreak prompts on large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, pages 1671–1685. ACM.

Wenju Sun, Qingyong Li, Wen Wang, Yang Liu, Yangliao Geng, and Boyang Li. 2025. Towards minimizing feature drift in model merging: Layer-wise task vector fusion for adaptive knowledge integration. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*.

Lintang Sutawika, Hailey Schoelkopf, Leo Gao, Baber Abbasi, Stella Biderman, Jonathan Tow, ben fattori, Charles Lovering, farzanehnakhaee70, Jason Phang, Anish Thite, Fazz, Aflah, Niklas Muennighoff, Thomas Wang, sdtblck, nopperl, gakada, tttyuntian, and 11 others. 2025. EleutherAI/lm-evaluation-harness: v0.4.8. Zenodo.

Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. 2022. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on*

Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA, volume 162 of *Proceedings of Machine Learning Research*, pages 23965–23998. PMLR.

Prateek Yadav, Derek Tam, Leshem Choshen, Colin A. Raffel, and Mohit Bansal. 2023. Ties-merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Kunda Yan, Min Zhang, Sen Cui, Zikun Qu, Bo Jiang, Feng Liu, and Changshui Zhang. 2025. CALM: consensus-aware localized merging for multi-task learning. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, volume 267 of *Proceedings of Machine Learning Research*, pages 70309–70329. PMLR / OpenReview.net.

Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. 2024. Adamerging: Adaptive model merging for multi-task learning. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.

Ziyi Yang, Fanqi Wan, Longguang Zhong, Tianyuan Shi, and Xiaojun Quan. 2025. Weighted-reward preference optimization for implicit model fusion. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net.

Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. 2024. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, volume 235 of *Proceedings of Machine Learning Research*, pages 57755–57775. PMLR / OpenReview.net.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*, pages 4791–4800. Association for Computational Linguistics.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, and Zheyan Luo. 2024. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pages 400–410. Association for Computational Linguistics.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. Instruction-following evaluation for large language models. *CoRR*, abs/2311.07911.

Yuhang Zhou, Giannis Karamanolakis, Victor Soto, Anna Rumshisky, Mayank Kulkarni, Furong Huang, Wei Ai, and Jianhua Lu. 2025. Mergeme: Model merging techniques for homogeneous and heterogeneous moes. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025 - Volume 1: Long Papers, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*, pages 2315–2328. Association for Computational Linguistics.

A Implementation Details

For CoMerge, we use an effective batch size of 16 (per-device batch size 1 with 16 gradient-accumulation steps), learning rate 10^{-2}, DPO $\beta = 0.1$, $\lambda_{\text{init}} = 0.4$, and 280 steps. We optimize the coefficients using AdamW with $(\beta_1, \beta_2) = (0.9, 0.999)$, $\epsilon = 10^{-8}$, zero weight decay, a cosine learning-rate schedule with a warmup ratio of 0.1, and a maximum gradient norm of 5.0; unless otherwise stated, the training seed is 42. Preference generation uses `do_sample=True`, temperature 0.7, top-p 0.9, and at most 1,024 new tokens. After global pruning, we apply `torch.svd_lowrank` with a target rank of 1500 to a projection matrix only when the factorized representation reduces storage; otherwise, the pruned matrix is retained. Normalization tensors are pruned but not SVD-factorized. Full-DPO uses 4 NVIDIA H20 GPUs with an effective batch size of 64 and is initialized from a Task Arithmetic merge of the globally Top-10%-pruned task vectors with a fixed merging coefficient of $\alpha = 0.4$. Unless otherwise noted, for the original MergeBench model-merging baselines, we use the best-performing model-specific configurations reported by MergeBench for the corresponding instruction-tuned backbones, as selected through grid search on surrogate validation tasks. Note that the Task Arithmetic results use the best-performing hyperparameter setting reported by MergeBench for the corresponding backbone, whereas the negative samples in $\mathcal{D}_{\text{pref}}$ are generated using a standard, uncalibrated Task Arithmetic model with fixed $\alpha = 0.4$.

Software Environment. We implement CoMerge as a custom extension of LLaMA-Factory (Zheng et al., 2024) 0.9.2.dev0 (upstream commit `b4c7dd3a`). Optimization experiments use Python 3.10.18, PyTorch 2.5.1+cu124, Transformers 4.45.2, TRL 0.9.6, Accelerate 0.34.2, Datasets 2.21.0, PEFT 0.12.0, and CUDA 12.4. General-capability evaluation uses `lm-evaluation-harness` (Sutawika et al., 2025) 0.4.8 with locally added task configurations in a Python 3.10.18 environment containing vLLM 0.8.3, PyTorch 2.6.0+cu124, Transformers 4.57.1, Accelerate 1.10.1, Datasets 4.2.0, and PEFT 0.17.1. HumanEval+ and MBPP+ use a local fork of the BigCode Evaluation Harness (Ben Allal et al., 2022) based on upstream commit `6116c6a` in the same evaluation environment. Safety evaluation uses a local `safety-eval` 1.0 fork based on upstream commit `2920bb85`, with vLLM 0.9.0.1, PyTorch 2.7.0+cu126, Transformers 4.53.3, Accelerate 1.10.1, Datasets 4.2.0, and PEFT 0.17.1. Complete package specifications and configuration files will be provided with the released code.

Evaluation Protocol. The general-capability harness evaluates instruction-following, mathematical, and multilingual tasks with the corresponding model-specific chat template. HumanEval+ and MBPP+ use the local BigCode Evaluation Harness fork, while the safety benchmarks use the local `safety-eval` fork. GSM8K uses 8-shot prompting and MATH uses 4-shot prompting; non-coding generation-based evaluations use greedy decoding. HumanEval+ and MBPP+ instead use temperature 0.2, top-p 0.95, and 10 sampled completions per problem, and report Pass@1. For each of M-MMLU, M-ARC, and M-HellaSwag, we evaluate the first 1,000 examples in the dataset loader's canonical order for each of French, Spanish, German, and Russian, giving 12 fixed language-specific tasks and 12,000 multilingual examples per model. The same subsets are used for all methods.

IFEval is scored by prompt-level loose accuracy; GSM8K by flexible-extraction exact match; MATH by exact match; M-MMLU by accuracy; and M-ARC and M-HellaSwag by normalized accuracy. XSTest uses overall accuracy, while DoAnything-Now macro attack success rate, WildGuardTest micro harm rate, and HarmBench micro attack success rate are converted to higher-is-better scores as one minus the corresponding rate. Each dataset score is normalized by the corresponding expert-model score. We then macro-average normalized scores within each capability and equally average the five capability scores to obtain Avg. NP. For Multilingual, we first average over the four languages within each dataset and then over the three datasets; the seven MATH subject scores are size-weighted before normalization.

Generative Adaptation of AdaMerging. The original AdaMerging (Yang et al., 2024) learns task-wise or layer-wise merging coefficients by minimizing the entropy of classifier outputs on unlabeled samples. The paper refers to its fine-grained variant as layer-wise merging, whereas the released implementation[2] instantiates these coefficients for individual parameter tensors. We therefore use

[2] https://github.com/EnnengYang/AdaMerging

tensor-wise to describe the implementation granularity of AdaMerging (Gen.). A causal language model instead produces a sequence of next-token distributions. We therefore use the current merged model to greedily decode a response

$$\tilde{\mathbf{y}} = \mathrm{GreedyDecode}\big(f_{\theta(\mathbf{\Lambda})}, x\big)\,, \tag{9}$$

where decoding is performed without gradient tracking. We then run a differentiable teacher-forced forward pass over the fixed prompt–response trajectory and minimize

$$\begin{aligned} h_t &= H\big(p_{\theta(\mathbf{\Lambda})}(\cdot \mid x, \tilde{\mathbf{y}}_{<t})\big)\,, \\ \mathcal{L}_{\mathrm{Ada}}(\mathbf{\Lambda}) &= \mathbb{E}_{x\sim\mathcal{D}_{\mathrm{cal}}}\left[\frac{1}{T_x}\sum_{t=1}^{T_x} h_t\right]. \end{aligned} \tag{10}$$

where $H(p) = -\sum_{v\in\mathcal{V}} p_v \log p_v$ is the Shannon entropy over the full vocabulary. The loss is averaged over all generated positions and then over samples, and only the merging coefficients are optimized. The chosen and rejected responses in the preference records are not used by this baseline.

Following the original AdaMerging learning-rate setup, we use a learning rate of 1×10^{-3}, zero weight decay, a constant schedule, and no warmup. We use Adam with $(\beta_1, \beta_2) = (0.9, 0.999)$ and a maximum gradient norm of 5.0. For a controlled comparison with CoMerge, AdaMerging (Gen.) uses the same 5,000 underlying calibration prompts, global Top-10% pruning with the same selective approximate SVD and target rank of 1500, coefficient initialization $\lambda = 0.4$, tensor-wise coefficient parameterization, effective batch size of 16, and one-epoch optimization budget. A 90/10 split yields 4,500 training and 500 validation prompts, corresponding to 281 optimizer steps. The maximum total sequence length is 512 for Llama-3.1-8B-Instruct and 1024 for the two smaller backbones. Model computation uses BF16, while the full-vocabulary entropy is evaluated in FP32. Because CoMerge and AdaMerging (Gen.) share this tensor-wise parameterization, both optimize $1{,}445$, $1{,}265$, and $1{,}435$ scalar coefficients for Llama-3.1-8B-Instruct, Llama-3.2-3B-Instruct, and Gemma-2-2b-it, respectively.

Multi-seed stability. For the two smaller backbones, we run CoMerge with seeds 42, 43, and 44 under the same pre-generated preference-pair pool and fixed settings, reporting the sample standard deviation.

Metric	Task Arithmetic	Base Model
Safety	**1.0340**	0.9759
Instruction	**0.9952**	0.8138
Math	0.9810	**0.9941**
Coding	1.0048	**1.0280**
Multilingual	**0.9687**	0.9365
Avg. NP	**0.9968**	0.9497

Table 6: Per-category negative-source control on Llama-3.1-8B-Instruct. The base model replaces Task Arithmetic only for generating rejected responses; prompts, chosen responses, pair count, decoding, coefficient parameterization, and optimization settings are fixed. **Bold** marks the better result.

B Additional Sweep for Iterative Preference Data

Table 7 reports a targeted sweep of iterative preference-data mixtures, with gains measured against $\mathcal{D}_1$. We expand along $\mathcal{D}_3$ for Llama-3.2-3B-Instruct and $\mathcal{D}_2$ for Gemma-2-2b-it, the more promising direction for each model. This probes gain boundaries and the necessity of retaining $\mathcal{D}_1$ rather than claiming an exhaustive search.

Mix $\mathcal{D}_1 : \mathcal{D}_2 : \mathcal{D}_3$	Avg. NP	Gain
Llama-3.2-3B-Instruct		
8:0:0	0.9706	–
8:2:0	0.9619	-0.0087
8:0:2	**0.9778**	**+0.0072**
8:1:1	0.9733	+0.0027
8:0:4	0.9701	-0.0005
8:0:8	0.9708	+0.0002
0:0:8	0.9644	-0.0062

Mix $\mathcal{D}_1 : \mathcal{D}_2 : \mathcal{D}_3$	Avg. NP	Gain
Gemma-2-2b-it		
8:0:0	0.9338	–
8:2:0	0.9358	+0.0020
8:0:2	0.9230	-0.0108
8:1:1	**0.9416**	**+0.0078**
8:4:0	0.9413	+0.0075
8:8:0	0.9399	+0.0061
0:8:0	0.8884	-0.0454

Table 7: Additional sweep and targeted sensitivity analysis for iterative preference-data mixtures.

C Additional Results

Method	Safety	Instruction	Math	Coding	Multilingual	Avg. NP
Llama-3.2-3B-Instruct						
Full-DPO	0.9779	**1.0024**	0.8792	0.9423	**1.0079**	0.9619
Data-free Methods						
Model Soup	0.9726	0.9121	0.9217	1.0055	0.9570	0.9538
Task Arithmetic	0.9666	0.8488	0.8714	1.0358	0.9771	0.9399
TIES-Merging	0.9710	0.9098	0.9324	1.0127	0.9729	0.9598
DARE	0.9621	0.8439	0.8790	1.0236	0.9788	0.9375
Dataless L&S	0.9672	0.9658	0.9522	0.9671	0.9721	0.9649
Data-driven Methods						
RegMean	0.9710	0.8926	0.9336	**1.0450**	0.9763	0.9637
RegMean++	0.9686	0.9024	0.9376	1.0387	0.9699	0.9634
Localize-and-Stitch	0.9764	0.9317	0.9525	0.9672	0.9928	0.9641
AdaMerging (Gen.)	0.9765	0.9609	0.9403	0.9407	0.9684	0.9574
CoMerge (Ours)	**0.9800**	0.9707	**0.9558**	1.0001	0.9466	**0.9706**
Gemma-2-2b-it						
Full-DPO	0.8870	**0.9839**	0.9340	0.8953	0.9473	0.9295
Data-free Methods						
Model Soup	0.9004	0.9572	0.8170	0.9431	0.9979	0.9231
Task Arithmetic	0.8970	0.9599	0.8130	**0.9442**	1.0003	0.9229
TIES-Merging	0.9012	0.9011	0.8200	0.9348	1.0309	0.9176
DARE	0.9163	0.8396	0.7832	0.9257	**1.0654**	0.9060
Dataless L&S	0.8876	0.8717	0.9301	0.9294	0.9606	0.9159
Data-driven Methods						
RegMean	0.8986	0.9278	0.8578	0.9361	1.0369	0.9314
RegMean++	0.8783	0.9706	0.8429	0.9393	1.0309	0.9324
Localize-and-Stitch	**0.9613**	0.6551	0.8257	0.8753	0.9812	0.8597
AdaMerging (Gen.)	0.8934	0.9064	0.8328	0.9340	0.9632	0.9060
CoMerge (Ours)	0.8997	0.9385	**0.9362**	0.9312	0.9634	**0.9338**

Three-seed stability of Avg. NP					
Backbone	**Seed 42**	**Seed 43**	**Seed 44**	**Best Baseline**	**Mean ± Std.**
Llama-3.2-3B-Instruct	0.9706	0.9665	0.9727	Dataless L&S (0.9649)	0.9699 ± 0.0032
Gemma-2-2b-it	0.9338	0.9314	0.9312	RegMean++ (0.9324)	0.9321 ± 0.0014

Table 8: Normalized MergeBench performance on Llama-3.2-3B-Instruct and Gemma-2-2b-it. The upper panel reports the single-run per-category comparison, where the CoMerge row corresponds to seed 42; within each backbone panel, the numerically best and second-best scores across all listed methods are shown in **bold** and underlined, respectively. The lower panel reports CoMerge Avg. NP across seeds 42, 43, and 44 (mean ± sample standard deviation).